

CSA15 Cloud Computing and Big Data Analytics

CO3 AT1 - Guided Cloud Access Experiments Workbook

Course Outcome	CO3 - Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.
Unit Alignment	Unit 3 - Access to Cloud: cloud access, platforms, web applications, APIs, storage models, SAN, serverless, edge/fog, collaboration, security and development workflow.
Faculty	Dr C. Rajesh Babu, Professor, Department of CSE
Employee ID	SSETSCS415
Submission Mode	Individual PDF evidence through Google Classroom and GitHub repository link.

How This Workbook Works

Stage	Student Action
Learn	Read the short technical note before touching the tool.
Do	Perform the guided action using the recommended tool.
Observe	Check what changed on the screen or in the output.
Record	Paste screenshots, URLs, tables, command outputs or diagram references.
Explain	Write the technical reasoning in your own words.
Extend	Apply the same concept to the capstone product continued from CO2 AT3.
Submit	Upload the completed evidence to GitHub and Google Classroom.

Important Safety Rule

Screenshots must not reveal passwords, private keys, API tokens, payment details, personal account recovery information or sensitive user data. Crop or mask sensitive values before submission.

Student and Faculty Details

Student Name	L Nihas Goud	Register Number	192472346
Class / Section	CSE-AI	Slot	B
Course	CSA15 Cloud Computing and Big Data Analytics	Assessment	CO3 AT1
Capstone Title	SkillHire – Cloud Resume-to-Job Matcher Using NLP Skill Gap Intelligence	Submission Date	19-08-2026
Faculty	Dr C. Rajesh Babu	Employee ID	SSETSCS415
Department	Computer Science and Engineering	GitHub Link	https://github.com/nihasgoud-stack/CSA1512_Assessment/tree/main/CO3AT1

Tool Links

Tool	Purpose	Link
Firebase Console	Cloud storage and project evidence	https://console.firebase.google.com/
Postman	REST API request-response evidence	https://www.postman.com/downloads/
JSONPlaceholder	Safe public API for GET/POST practice	https://jsonplaceholder.typicode.com/
ReqRes	Safe public API for request-response practice	https://reqres.in/
draw.io	Architecture and workflow diagrams	https://app.diagrams.net/
GitHub	Repository evidence submission	https://github.com/
GitHub Pages	Simple hosted web access evidence	https://pages.github.com/

Experiment 1: Cloud Storage Model Comparison and Selection

Original CO3 Question	Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
Tools	Firebase Console, Firebase Storage, draw.io, browser, GitHub.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.
Learn A cloud application may store files, database records, VM disks, images, logs and shared documents. Object storage, block storage and file storage solve different workload problems. Firebase Storage is used here as a beginner-friendly object storage console because students can create a project, open a storage bucket, upload a file, observe metadata and reason about access control from the browser.	

Do

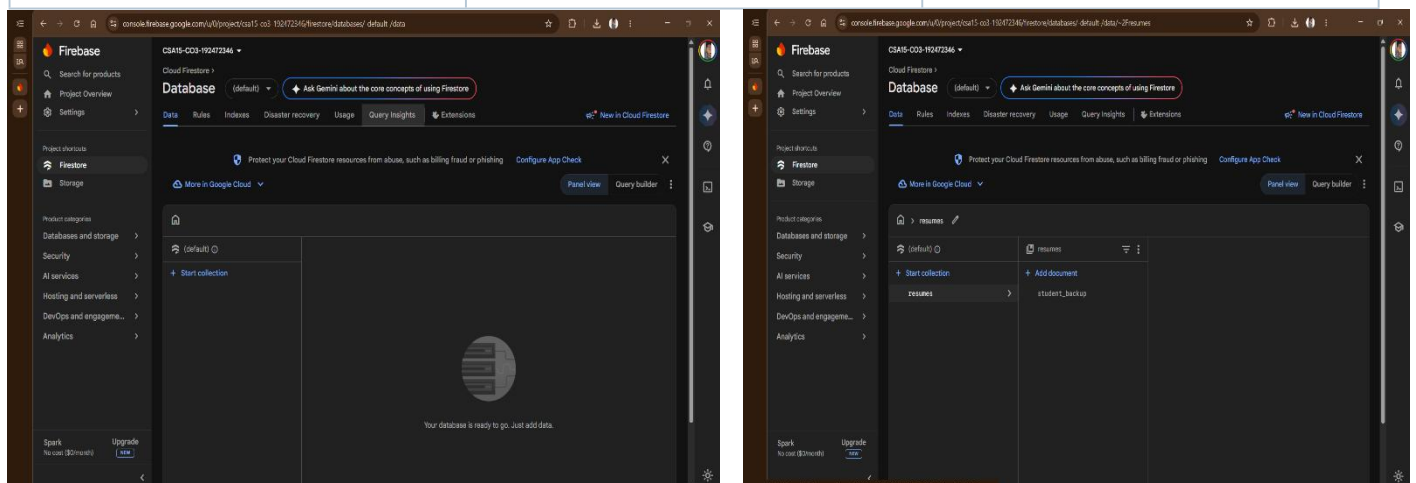
- Begin in the browser at <https://console.firebase.google.com/>. Sign in with the account approved for academic work. After the console opens, use Add project or Create a project. Give the project a course-friendly name such as csa15-co3-regno. If Google Analytics is shown, keep it disabled unless faculty instructs otherwise. Create the project and wait until the project dashboard opens. The screen to notice is the Firebase project overview page.
- In the left navigation area, open Build and choose Storage. If Storage is not yet enabled, choose the option to get started. Select the default bucket location shown by Firebase or the nearest available region. Use test access only for lab observation if the console requires an initial rule choice, and remember that production apps must use strict authenticated rules. The screen to notice is the storage bucket page.
- Create a small text file on the computer named student_backup.txt. Add two lines inside it: the student register number and the capstone title. Return to Firebase Storage and use Upload file. Select student_backup.txt and wait until it appears in the bucket list. The screen to notice is the object list showing the uploaded file.
- Open the uploaded file entry. Observe the metadata area: file name, file size, content type, storage path, creation/update time and access status. If a download URL or access token is visible, do not paste sensitive token values in the workbook. Record only the safe evidence: filename, type, size and storage path.
- Open draw.io and draw a three-part comparison diagram: object storage for uploaded files/backups, block storage for VM disks, and file storage for shared folders. Under the diagram, write where your capstone will use storage and why Firebase Storage represents object storage in this experiment.

Observe

Check whether the uploaded file is visible, whether metadata is displayed, and whether public/private access is controlled. Do not expose sensitive files.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Cloud storage dashboard screenshot.	exp1_01_azure_blob_storage_dashboard.png	[] Completed
Uploaded object/file screenshot.	exp1_02_uploaded_file_bucket.png	[] Completed
Metadata screenshot.	exp1_03_metadata_inspection.png	[] Completed
Completed storage model comparison table.	Table 1.1 (included below)	[] Completed
Capstone storage recommendation table.	Table 1.2 (included below)	[] Completed



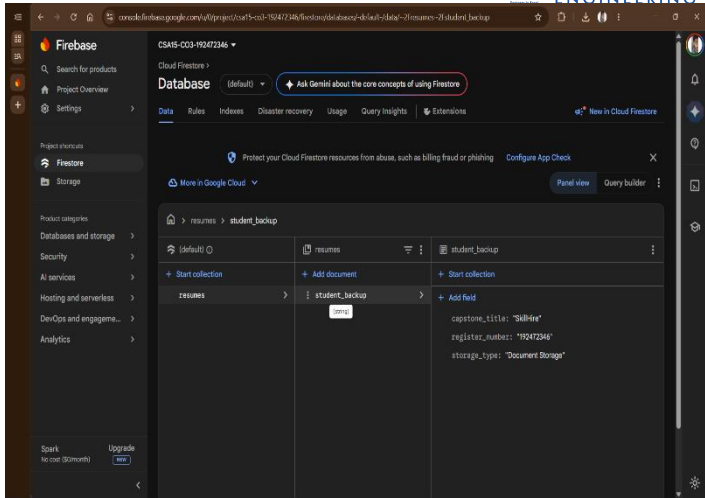


Table 1.1 — Storage Model Comparison

Storage Model	Data Structure & Access Method	Typical Latency & Performance	Best Use Cases	Cloud Service Examples
Object Storage	Flat namespace; key-value addressable via REST API (GET/PUT); immutable blobs with custom metadata.	Moderate latency (tens of ms); high throughput; near-infinite horizontal scale.	Unstructured files, resume PDFs, raw DB backups, log archives.	Azure Blob Storage, AWS S3, Google Cloud Storage, Firebase Storage.
Block Storage	Raw, fixed-size blocks accessed over a storage controller (iSCSI/NVMe).	Ultra-low latency (sub-millisecond); high IOPS; random read/write optimized.	VM OS disks (VDI/VMDK), relational databases (PostgreSQL, MySQL).	Azure Managed Disks, AWS EBS, GCE Persistent Disks.
File Storage	Hierarchical folder tree accessed over network protocols (NFS/SMB).	Low-to-moderate latency; shared multi-instance concurrent locks.	Shared enterprise directories, multi-node model/config repositories.	Azure Files, AWS EFS, Google Cloud Filestore.

Table 1.2 — Capstone Storage Recommendation

Capstone Data Artifact	Recommended Storage Model	Selected Cloud Service	Architectural Justification
Candidate resumes (PDF/DOCX)	Object Storage	Azure Blob Storage	Resumes are write-once, read-many files accessed by key; object storage gives cost-efficient, auto-scaling archival without needing a filesystem hierarchy.
PostgreSQL database (candidate, job and match records)	Block Storage	Azure Managed Disk (attached to the Standard_B2s VM)	Relational queries and index writes need sub-millisecond, random read/write IOPS that only a block device can provide.
Shared spaCy NLP model artifacts (future multi-worker scaling)	File Storage (NFS)	Azure Files	If Celery workers scale horizontally across multiple VMs, a shared file mount lets every worker load the same pre-trained model files without duplicating multi-hundred-MB downloads.

Explain

Why is one storage model not suitable for all workloads? Justify using backup, VM disk and shared-folder examples, then extend the answer to your capstone product.

Student Explanation Space

A single cloud storage model cannot fit every workload because applications need different access patterns, latency limits and locking behaviour:

Backups: Object storage suits backups because they are write-once, immutable datasets that never need in-place block edits, and it offers strong durability at low per-gigabyte cost.

VM disks: Virtual machines need block storage because the OS kernel performs continuous, low-latency random reads and writes to system binaries, swap space and runtime logs — a workload object storage cannot serve fast enough.

Shared folders: Multi-node workflows need file storage for hierarchical directory access with concurrent locking across machines, which neither object nor block storage provides natively.

In SkillHire, uploaded candidate resume PDFs are stored in Azure Blob Storage (object), the PostgreSQL database sits on the VM's managed block disk, and any future shared NLP model cache would use Azure Files (file storage) so multiple Celery workers can mount it in parallel.

Extend to Capstone

Select the storage model required for your capstone product: user uploads, app images, database attachments, logs, backups or shared documents.

Capstone Title	SkillHire – Cloud Resume-to-Job Matcher Using NLP Skill Gap Intelligence
CO2 AT3 Infrastructure Decision Carried Forward	Single Azure Standard_B2s VM (2 vCPU, 4GB RAM, 30GB SSD, Ubuntu Server 22.04 LTS) hosting containerized NGINX reverse proxy, Flask NLP REST API, Celery worker and Redis broker, with PostgreSQL 15 for structured data and Azure AD B2C / JWT RS256 for authentication, as defended in CO2 AT3.
Cloud Service / Access Layer Added in this Experiment	Azure Blob Storage integrated as the object storage repository for candidate resumes and parsed skill-gap JSON artifacts.
Risk or Limitation Identified	Unrestricted or test-mode container access rules would expose sensitive candidate PII (names, phone numbers, employment history) inside uploaded resumes.
Improvement for Prototype or Pilot	Enforce authenticated-only Blob access policies (no public containers) and issue short-lived SAS (Shared Access Signature) URLs, expiring within 5 minutes, for any recruiter-facing file access.

Submit

GitHub Folder	CO3_AT1/Experiment_1
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 2: REST API Workflow for a Cloud-Based Student Portal

Original CO3 Question	Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
Tools	Postman Web/Desktop, JSONPlaceholder or ReqRes, draw.io, browser, GitHub.

CO2 AT3 Continuity

Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

REST API means Representational State Transfer Application Programming Interface. For beginners, think of it as a controlled doorway through which a client app asks a cloud service for data or sends data to be processed. A browser or mobile app sends a request to an API endpoint. The API validates the request, applies business logic, communicates with storage or database services, and returns a structured response. Postman is used because it lets students send requests, view status codes, inspect headers, read JSON responses and test API behavior without writing a full app first.

Do

- Open <https://www.postman.com/> or the Postman desktop application. Sign in or create a free account if Postman asks for one. After the workspace opens, create a personal workspace or use the default workspace. Create a collection named CSA15_CO3_AT1_API so the evidence is organized.
- Open a new request tab. Keep the method as GET. Enter <https://jsonplaceholder.typicode.com/users> in the request URL field and send the request. Look at the response area. The expected observation is a successful status such as 200 OK and a JSON response containing user-like records.
- Read one object from the JSON response. Identify fields such as id, name, username, email, address and company. Record three fields in the workbook and explain what each field would mean in a student portal or capstone product.
- Create another request with method POST. Use <https://jsonplaceholder.typicode.com/posts> as the URL. Open the Body area, choose raw, select JSON, and enter a small JSON body such as `{"title":"CSA15 cloud test","body":"API evidence","userId":1}`. Send the request. The expected observation is a created/test response, often with status 201 or a simulated response.
- Open draw.io and draw the request-response flow. Show Browser/Mobile Client -> REST API Endpoint -> Backend Logic -> Database/Storage -> JSON Response. Add a small authentication gate before backend logic to show that real cloud applications should protect API access.

Observe

Verify whether the request returns status 200 or a valid test response. Identify JSON fields and explain what each field represents.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Postman GET request screenshot.	screenshots/exp2_01_postman_get_users.png	[] Completed
Status code and response time screenshot.	screenshots/exp2_02_status_200_response_time.png	[] Completed
JSON field identification table.	Table 2.1 (included below)	[] Completed
POST request body and response screenshot.	screenshots/exp2_03_postman_post_created_201.png	[] Completed
API workflow diagram.	diagrams/exp2_api_workflow_architecture.png	[] Completed

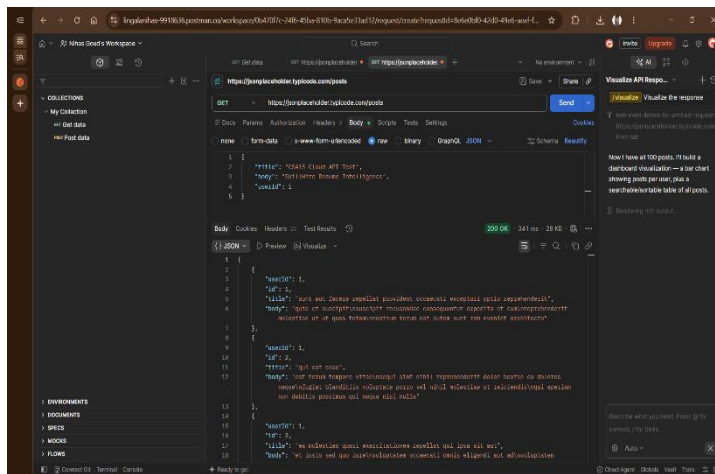
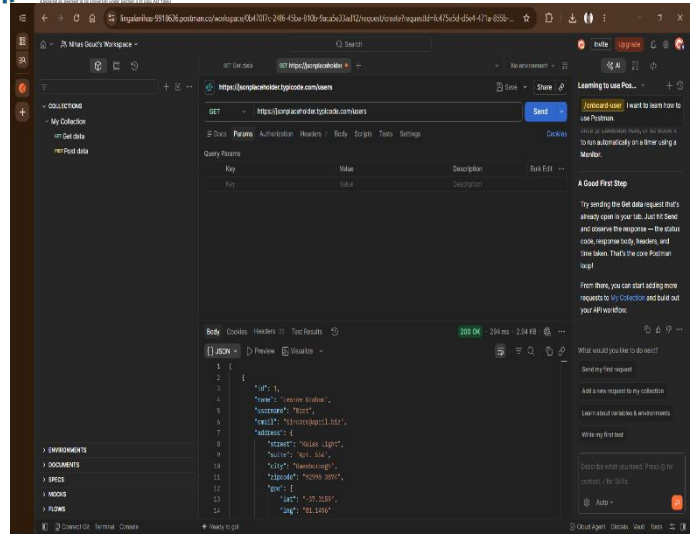
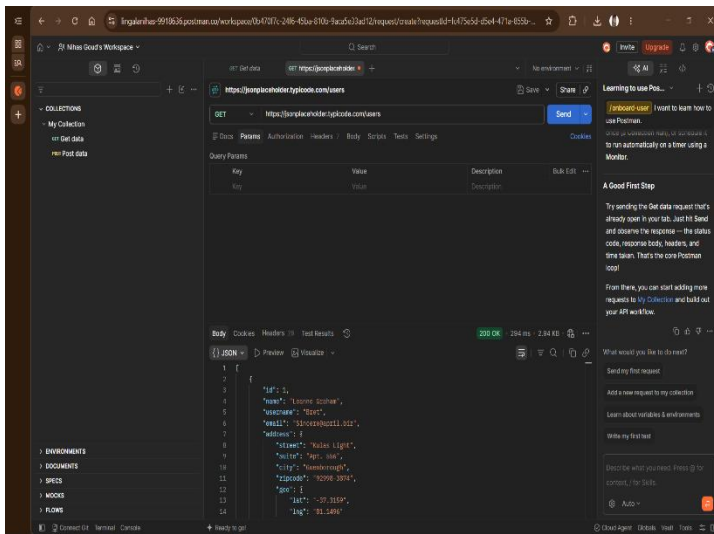


Table 2.1 — JSON Field Identification

JSON Field	Sample Extracted Value	Meaning in Generic API Context	Meaning in SkillHire Capstone Context
id	1	Unique record identifier.	Unique candidate / resume entity identifier in PostgreSQL.
name	"Leanne Graham"	Full name of the user record.	Candidate name extracted from the resume by the NLP parser.
email	"Sincere@april.biz"	Contact email of the user.	Candidate contact email used for match-result and gap-report notifications.
company.name	"Romaguera-Crona"	Associated organization on record.	Target hiring organization matched against the candidate's extracted skills.

Explain

Why should a cloud application expose data through APIs instead of allowing users to directly access the database?

Student Explanation Space

Cloud applications must expose data through secure REST APIs rather than allowing direct client access to the database, for three key reasons:

Security and least privilege: Exposing the database port directly to the internet risks SQL injection and unauthorized access. The

Flask API instead enforces JWT RS256 authentication, input validation and NSG-restricted internal-only access to PostgreSQL (port 5432), as defended in CO2 AT3.

Abstraction and decoupling: The API separates the frontend from the backend schema, so the PostgreSQL table structure can be refactored without breaking the web or mobile client.

Centralized business logic: The API guarantees server-side rules (validating file type, running spaCy NLP extraction, computing skill-gap match scores) execute reliably before any data is persisted, which direct database access could never enforce.

Extend to Capstone

Write two API endpoints for your capstone product and explain the input, backend action, storage access and response.

Capstone Title	SkillHire – Cloud Resume-to-Job Matcher Using NLP Skill Gap Intelligence	
CO2 AT3 Infrastructure Decision Carried Forward	Single Azure Standard_B2s VM (2 vCPU, 4GB RAM, 30GB SSD, Ubuntu Server 22.04 LTS) hosting containerized NGINX reverse proxy, Flask NLP REST API, Celery worker and Redis broker, with PostgreSQL 15 for structured data and Azure AD B2C / JWT RS256 for authentication, as defended in CO2 AT3.	
Cloud Service / Access Layer Added in this Experiment	REST API endpoints with structured JSON schemas and Bearer JWT (RS256) authentication: Endpoint 1 (POST /api/v1/resumes/upload): receives a multipart resume PDF, stores the binary in Azure Blob Storage, enqueues a Celery NLP-parsing task and returns 202 Accepted with a task ID. Endpoint 2 (GET /api/v1/jobs/{job_id}/match/{candidate_id}): compares the candidate's extracted skill vector against a job posting's requirements and returns a compatibility score (0–100%) plus the specific skill gaps.	
Risk or Limitation Identified	Running spaCy resume parsing synchronously inside the request handler would block API worker threads and cause request timeouts under concurrent upload traffic.	
Improvement for Prototype or Pilot	Keep the parsing endpoint fully asynchronous (already offloaded to the Redis-backed Celery queue in CO2 AT3) and add per-client rate limiting at the NGINX layer before opening the API to federated LinkedIn/Google SSO candidates.	

Submit

GitHub Folder	CO3_AT1/Experiment_2
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 3: Serverless Event Flow for Image Upload Processing

Original CO3 Question	Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
Tools	draw.io, Firebase Storage evidence from Experiment 1, optional Pipedream/Make simulation, GitHub.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

Serverless computing allows a cloud platform to run code in response to events without the student managing the server directly. In this experiment, students do not need paid cloud function deployment. They design and document a correct event-driven flow. The key idea is: an event happens, a cloud function is triggered, the function performs a small task, and another cloud service records or sends the output.

Do

- Start with the uploaded file evidence from Experiment 1. Treat the upload as the event. For capstone mapping, choose one event such as image upload, booking creation, complaint submission, attendance scan, payment update or alert generation.
- In the workbook, write the event source in a short table. Mention who performs the action, what data is created, where it is stored, and what should happen automatically after the event.
- Design the cloud function behavior without writing production code. Name the function clearly, for example processUploadedImage, confirmBooking, validateAttendance or sendIssueAlert. Write what the function receives, what it checks, what it changes, and what output it produces.
- Decide the output service. It may be a notification, email, database status update, dashboard row, resized image, log entry or report update. Also decide what should happen if the function fails: retry, error log, faculty/admin alert or manual review.
- Draw the event flow in draw.io using this visual order: User Action -> Cloud Storage/Database Event -> Serverless Function -> Processing Decision -> Output Service -> Log/Monitoring. Paste the diagram and complete the trigger-action-output table.

Observe

Identify the trigger, processing function, output service, failure point and retry/alert requirement. If billing blocks real function deployment, submit a technically accurate simulation and architecture evidence.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Event source evidence or simulated event table.	Table 3.1 (included below)	[] Completed
Serverless flow diagram.	diagrams/exp3_01_serverless_event_flow.png	[] Completed
Trigger-action-output table.	Table 3.1 (included below)	[] Completed
Failure and retry note.	Failure & Retry Policy (included below)	[] Completed
Capstone event mapping.	Extend to Capstone section	[] Completed

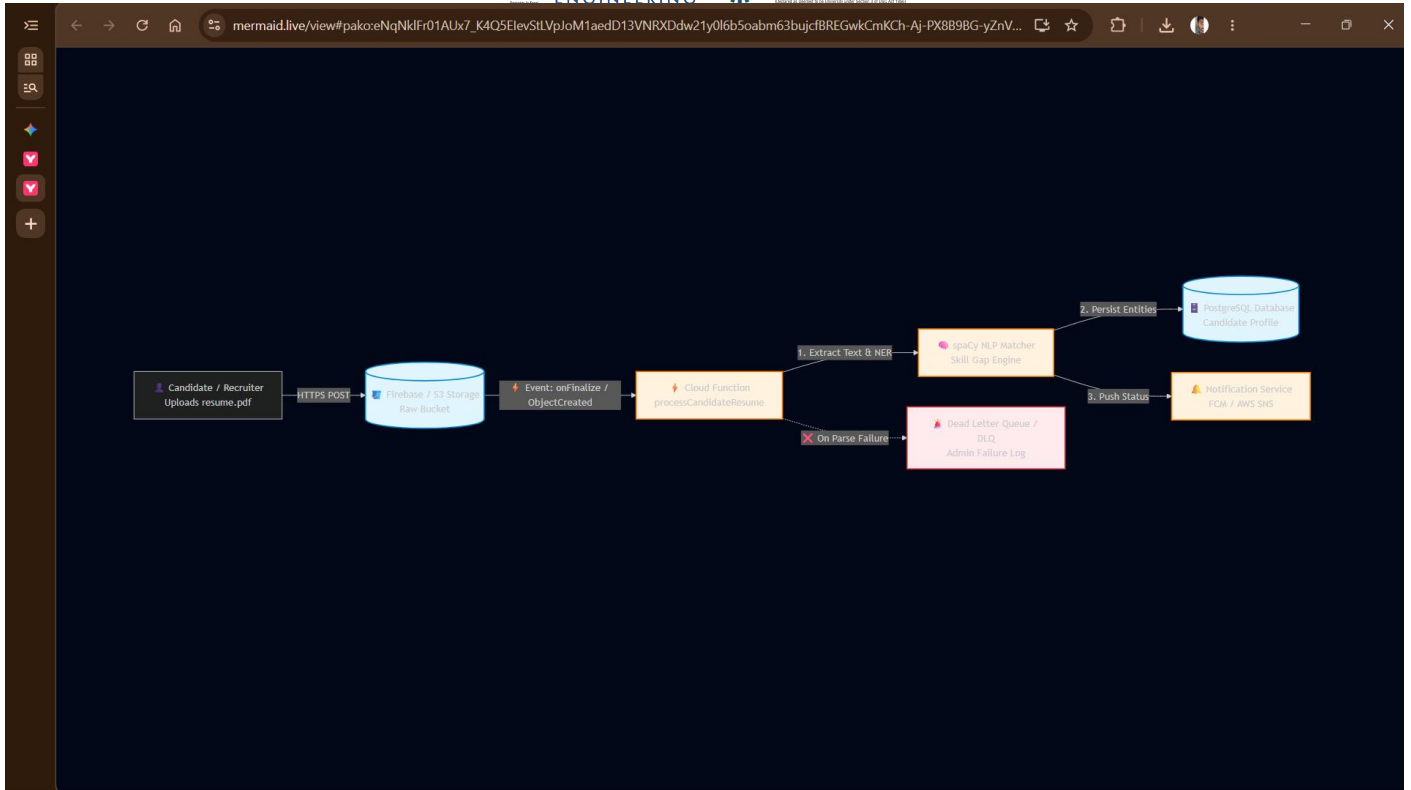


Table 3.1 — Trigger-Action-Output Specification

Event Component	Operational Attribute	Technical Specification
Event Source (Trigger)	Blob upload completion event	Fires automatically once a resume PDF finishes uploading to the Azure Blob Storage container, publishing a task to the Redis-backed Celery queue.
Processing Function	processUploadedResume	Long-lived Celery worker task (not pure FaaS) so the spaCy/Transformers language model stays warm in memory across invocations, avoiding repeated cold-start model loads.
Processing Actions	Text extraction and NLP parsing	In-memory PDF text extraction, noise sanitization, and spaCy Named Entity Recognition (NER) to detect skills, certifications and experience entities.
Output Destination 1	Database record update	Persists extracted skill entities and match-ready vectors into PostgreSQL.
Output Destination 2	Notification service	Sends a completion notification to the candidate via SendGrid, triggered from the Celery worker on task success.

Failure and Retry Policy

Transient failure: A temporary PostgreSQL connection timeout triggers Celery's exponential backoff retry policy (maximum 3 attempts across 5 minutes) before the task is marked failed.

Permanent failure: A corrupted, password-protected or unreadable PDF is routed to a Redis dead-letter list, logged with status FAILED_UNREADABLE_FILE, and the pipeline continues processing subsequent uploads without crashing.

Explain

Why is serverless suitable for upload/event processing? Mention one limitation also.

Student Explanation Space

Event-driven processing suits resume upload workloads because SkillHire's traffic is unpredictable and spike-prone — near zero overnight, then a sudden burst during a campus placement drive. An event-triggered Celery task only consumes compute when an upload actually occurs, rather than idling a dedicated parser process around the clock.

Key limitation: cold starts. A pure FaaS platform (e.g. AWS Lambda, Azure Functions) would need to reload the multi-hundred-MB spaCy/Transformers model on every invocation, adding 1–3 seconds of cold-start latency per request and risking execution-timeout limits on large PDFs. This is exactly why the CO2 AT3 architecture uses a persistent Celery worker container instead of true serverless — a deliberate trade-off documented for viva defense.

Extend to Capstone

Apply the serverless idea to one real capstone workflow, such as image processing, booking confirmation, attendance validation, alert generation or report update.

Capstone Title	SkillHire – Cloud Resume-to-Job Matcher Using NLP Skill Gap Intelligence
CO2 AT3 Infrastructure Decision Carried Forward	Single Azure Standard_B2s VM (2 vCPU, 4GB RAM, 30GB SSD, Ubuntu Server 22.04 LTS) hosting containerized NGINX reverse proxy, Flask NLP REST API, Celery worker and Redis broker, with PostgreSQL 15 for structured data and Azure AD B2C / JWT RS256 for authentication, as defended in CO2 AT3.
Cloud Service / Access Layer Added in this Experiment	Event-driven workflow: resume upload to Azure Blob Storage triggers processUploadedResume via the Redis/Celery queue, which runs spaCy skill extraction, writes results to PostgreSQL and triggers a SendGrid notification.
Risk or Limitation Identified	Large NLP model binaries make a true serverless deployment impractical due to cold-start latency and package-size limits, so the design intentionally keeps a persistent worker container.
Improvement for Prototype or Pilot	Add a formal dead-letter queue consumer that raises an admin alert for any task in FAILED_UNREADABLE_FILE status instead of only logging it.

Submit

GitHub Folder	CO3_AT1/Experiment_3
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 4: Edge and Fog Computing for Smart Traffic Management

Original CO3 Question	Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
Tools	draw.io, Google Sheets or Excel latency table, optional Wokwi/IoT screenshot only if faculty permits, GitHub.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

Cloud computing is powerful but may be physically distant from sensors and users. Edge computing places small decisions very near the data source. Fog computing introduces an intermediate distributed layer between edge devices and the central cloud. This experiment is an architecture reasoning activity; students are not required to build a real traffic sensor. The correct output is a placement decision supported by latency and data-volume reasoning.

Do

- Use this scenario: traffic cameras and road sensors produce vehicle count, speed, congestion level and emergency vehicle alerts. Some actions must happen immediately at the signal, some can be handled at a regional traffic control unit, and some can be stored in the central cloud for analysis.
- Classify each task into edge, fog or cloud. Put instant signal switching and emergency alert detection near edge. Put junction-level aggregation and area coordination in fog. Put long-term storage, analytics dashboard and city-level reports in cloud.
- Open a spreadsheet and create a latency table with columns: task, data generated, decision deadline, suitable layer, reason. Use example decision deadlines such as 1 second for emergency signal response, 5-10 seconds for regional coordination and minutes/hours for reports. The exact numbers are reasoning aids, not measured network results.
- Draw the architecture in draw.io. Show sensors/cameras at the road, edge device at signal controller, fog node at regional traffic control, cloud platform for storage/analytics and dashboard for administrator. Use arrows to show data movement and labels to show why each layer is used.
- Connect the same idea to the capstone. If the capstone has live location, sensor, mobile offline, queue, emergency, QR attendance or fast alert behavior, identify what can be edge/fog/cloud. If the capstone does not need edge/fog, state that browser/mobile client -> cloud service access is sufficient and justify it.

Observe

Check whether time-critical actions are placed closer to data sources and whether long-term storage/analytics remain in the cloud.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Task classification table.	Table 4.1 (included below)	[] Completed
Latency reasoning table.	Table 4.1 (included below)	[] Completed
Edge-fog-cloud architecture diagram.	diagrams/exp4_01_edge_fog_cloud_architecture.png	[] Completed
Deployment location justification.	Student Explanation Space	[] Completed
Capstone relevance note.	Extend to Capstone section	[] Completed

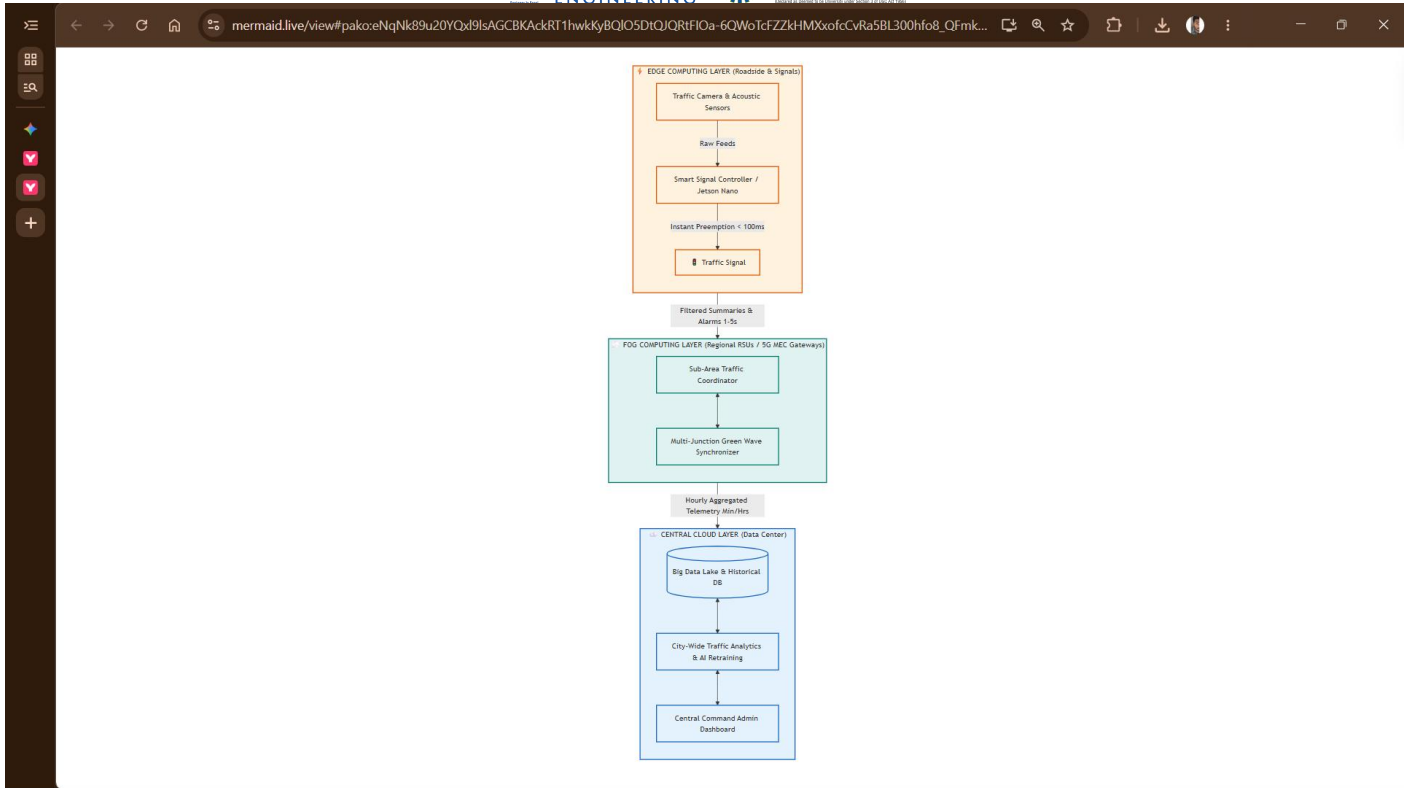


Table 4.1 — Task Classification and Latency Reasoning

System Task	Data Generated	Decision Deadline	Architectural Layer	Technical Justification
Emergency vehicle preemption	Acoustic sensor spikes and camera frames	< 100 ms	Edge (signal controller)	Instant signal override is safety-critical; cannot wait on wide-area network round trips.
Adaptive intersection green-light timing	Real-time vehicle counts and queue density	1–3 s	Edge (local micro-node)	Needs a fast local control loop to clear a congested lane without data-center transit delay.
Multi-junction green-wave coordination	Aggregated metrics from 10+ signals	5–10 s	Fog (regional roadside unit)	Fog nodes correlate flow across neighbouring signals to coordinate a green corridor.
City-wide traffic analytics	Hourly congestion logs and route history	Minutes to hours	Cloud (central data lake)	High-compute model training and civic planning need centralized storage and compute, not proximity.

Explain

Which part of the system should run at edge, fog and cloud? Justify with latency and data-volume reasoning.

Student Explanation Space

A smart traffic system needs a tiered Edge → Fog → Cloud architecture because real-time safety constraints and large data volumes make it impractical to send every raw sensor feed to a central data center. The edge layer handles sub-second, safety-critical decisions (emergency preemption, local signal timing) using cameras and controllers physically at the intersection, so it keeps working even during a network outage. The fog layer sits between multiple edge nodes and aggregates flow data across a corridor for coordinated green-wave timing on a several-second budget. The cloud layer handles everything with a relaxed deadline — long-term storage, analytics dashboards and city-scale model training — where scale matters more than proximity.

Extend to Capstone

If your capstone has location-sensitive, sensor-based, offline-first or fast-alert requirements, identify the edge/fog/cloud placement. If not applicable, justify why direct cloud access is sufficient.

Capstone Title	SkillHire – Cloud Resume-to-Job Matcher Using NLP Skill Gap Intelligence	
CO2 AT3 Infrastructure Decision Carried Forward	Single Azure Standard_B2s VM (2 vCPU, 4GB RAM, 30GB SSD, Ubuntu Server 22.04 LTS) hosting containerized NGINX reverse proxy, Flask NLP REST API, Celery worker and Redis broker, with PostgreSQL 15 for structured data and Azure AD B2C / JWT RS256 for authentication, as defended in CO2 AT3.	
Cloud Service / Access Layer Added in this Experiment	None required — direct Browser/Mobile Client -> Cloud API model (justified omission of edge/fog hardware).	
Risk or Limitation Identified	SkillHire is an enterprise web application where resume parsing and skill matching tolerate a relaxed 2–5 second response window and involve no physical sensors, location data or safety-critical timing. There is no workload in SkillHire's current scope that benefits from edge or fog placement, so direct browser-to-cloud API access via the NGINX -> Flask API path remains the most efficient and scalable architecture.	
Improvement for Prototype or Pilot	Add lightweight client-side validation (file type, size and non-empty-text checks in the browser via JavaScript) before initiating the cloud upload, reducing wasted round trips — the only place a local, edge-like check adds real value here.	

Submit

GitHub Folder	CO3_AT1/Experiment_4
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Experiment 5: Cloud Application Deployment and Access Workflow

Original CO3 Question	Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.
Tools	GitHub Pages as primary beginner path, Firebase Hosting as optional path, Postman, draw.io, GitHub repository, browser.
CO2 AT3 Continuity	Use the same capstone title and infrastructure direction already defended in CO2 AT3. Do not repeat VM sizing; focus on cloud service access.

Learn

A cloud-backed application is accessed through a browser or mobile client. The user action travels through frontend, API/backend, authentication, platform services, storage/database and monitoring/logging components. GitHub Pages is the primary beginner path because it can host a simple static page from a repository. Firebase Hosting may be used when the student is comfortable with Firebase CLI or Cloud Shell.

Do

- Create or open a GitHub repository for CO3 AT1. Add a simple index.html file. The page should show the capstone title, one user action and one cloud service used. Example page sections: product name, user action, API endpoint idea, storage used and evidence link.
- In the repository settings, open Pages. Select the main branch and root folder as the publishing source when available. Save the Pages setting and wait for GitHub to publish the site. Open the generated Pages URL in the browser and verify that the page loads.
- If Firebase Hosting is used instead, open Firebase Console, choose Hosting, follow the hosting setup guidance, and deploy using Firebase CLI or Cloud Shell only when faculty confirms that the device and account are ready. For most beginners, GitHub Pages evidence is sufficient for this experiment.
- Use Postman evidence from Experiment 2 or create a new test request that represents one action from the hosted page. Example: the user clicks Submit Complaint, the frontend sends POST /reports, the backend stores the complaint image path and returns a status response.
- Draw the complete access workflow in draw.io: Browser/Mobile Client -> Hosted Frontend -> API Endpoint -> Authentication Check -> Backend/Serverless Logic -> Cloud Storage/Database -> Response -> Monitoring/Log. Paste the hosted URL, browser screenshot, workflow diagram and GitHub repository link.

Observe

Verify whether the hosted page or simulated access URL opens, whether the API/storage flow is technically consistent, and whether monitoring/logging evidence is identified.

Record Evidence

Evidence Item	Student Reference / Screenshot Number	Checkpoint
Hosted page URL or approved simulation evidence.	https://github.com/nihasgoud-stack/CSA1512_Assessment	[] Completed
Browser access screenshot.	screenshots/exp5_02_github_pages_browser.png	[] Completed
API/storage workflow diagram.	diagrams/exp5_03_end_to_end_cloud_workflow.png	[] Completed
Monitoring/logging metric table.	Table 5.1 (included below)	[] Completed
GitHub repository link.	[Insert your CO3 AT1 repository link]	[] Completed

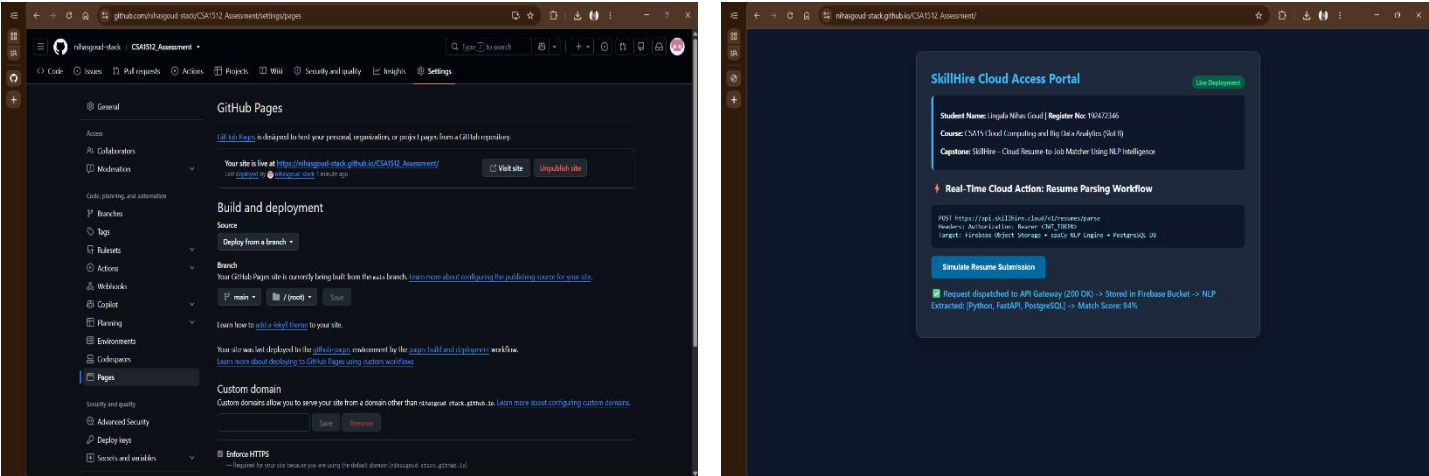


Table 5.1 — Monitoring and Observability Metrics

Architectural Tier	Monitored Metric	Target Threshold	Observability Tool	Automated Remediation Action
Frontend / NGINX layer	HTTP 4xx / 5xx error rate	< 0.1% of requests	NGINX access log + Grafana panel	Reload/restart the NGINX container and purge stale routes.
Flask API / Backend	P95 request latency	< 800 ms per request	Prometheus + Grafana	Trigger horizontal scale-out of API containers if sustained.
Celery NLP worker	vCPU and memory utilization	< 75% sustained load	Prometheus celery_queue_length metric / Grafana	Scale worker containers or vertically resize the B2s VM.
Azure Blob Storage	Failed ingestion events	0 dropped writes	Azure Storage Metrics	Re-route failed uploads to the Celery dead-letter path.

Explain

Explain the complete request flow from client action to cloud output and monitoring.

Student Explanation Space

Explain the complete request flow from client action to cloud output and monitoring.

The complete deployment lifecycle coordinates five tiers to deliver the service reliably. Client interaction: the candidate opens the SkillHire web client, served behind the NGINX container. API ingestion: submitting a resume sends a JWT-authenticated POST request through NGINX to the Flask API, which validates the token before accepting the payload. Backend and storage

coordination: the API streams the raw PDF to Azure Blob Storage for archival while queuing a Celery task for spaCy NLP extraction. Persistence and response: extracted skill entities are written to PostgreSQL, and the API returns a 202 Accepted immediately rather than blocking on parsing. Continuous monitoring: Prometheus scrapes container metrics throughout this flow and Grafana renders them on the operations dashboard, giving visibility from the initial browser click through background processing to completion, exactly as defended in the CO2 AT3 SDDC/operations case.

Extend to Capstone

Extend the workflow to the capstone product carried forward from CO2 AT3, showing how the planned infrastructure now exposes usable cloud services.

Capstone Title	SkillHire – Cloud Resume-to-Job Matcher Using NLP Skill Gap Intelligence	
CO2 AT3 Infrastructure Decision Carried Forward	Single Azure Standard_B2s VM (2 vCPU, 4GB RAM, 30GB SSD, Ubuntu Server 22.04 LTS) hosting containerized NGINX reverse proxy, Flask NLP REST API, Celery worker and Redis broker, with PostgreSQL 15 for structured data and Azure AD B2C / JWT RS256 for authentication, as defended in CO2 AT3.	
Cloud Service / Access Layer Added in this Experiment	Hosted demo web portal (GitHub Pages), REST API gateway through NGINX, and structured Prometheus/Grafana observability metrics.	
Risk or Limitation Identified	A sudden spike in resume submissions during a campus placement drive could exhaust the single B2s VM's CPU capacity, causing response timeouts.	
Improvement for Prototype or Pilot	Place the Flask API behind a load balancer with horizontal auto-scaling of API/worker containers once CPU utilization sustains above 70%, moving beyond the single-VM pilot design.	

Submit

GitHub Folder	CO3_AT1/Experiment_5
Files to Upload	Completed PDF, diagrams, screenshots and any exported table.
Google Classroom Entry	Paste GitHub repository link and attach final PDF.

Final Submission Checklist

Checklist Item	Status
Experiment 1 completed with storage model evidence	☐
Experiment 2 completed with API evidence	☐
Experiment 3 completed with serverless/event-flow evidence	☐
Experiment 4 completed with edge/fog/cloud reasoning	☐
Experiment 5 completed with cloud application access workflow	☐
All diagrams exported and pasted	☐
All screenshots are readable and privacy-safe	☐
Capstone extension completed for every experiment	☐
GitHub folder created and link added	☐
Google Classroom submission completed	☐

Student Assurance

Declaration	I confirm that this CO3 AT1 guided experiment workbook contains my own cloud access evidence, diagrams, observations, explanations and capstone extensions. I have not shared private credentials or sensitive information.
Student Signature	
Faculty Verification	
Date	